

Scientific AI Agents Must Learn and Share

Thomas Pan
Alpha Mind Club, LLC
Birmingham, AL, USA
tpan.6048@gmail.com

Jake Y. Chen
Department of Biomedical Informatics and Data Science
University of Alabama at Birmingham
Birmingham, AL, USA
jakechen@uab.edu

Abstract—Scientific artificial intelligence (AI) workflow agents have been developed to plan, execute, and evaluate tasks. Here, our study shows that they must also learn before execution and share after assessment to become efficient. Current Plan–Execute–Evaluate loops acquire domain knowledge reactively through tool calls and discard synthesized results at session end. These two architectural gaps create predictable costs: context grows with the volume of iterative tool calls, and knowledge produced in one run cannot be reused by subsequent agents without rediscovery. We propose PLEAS (Plan, Learn, Execute, Assess, Share), a substrate-independent five-phase specification that fills both gaps through an explicit Learn phase — structured knowledge retrieval before any tool is invoked — and a Share phase — artifact externalization after assessment. Five structural forces, formally defined, produce a context-bound that grows with task complexity parameters rather than with session length. We validate PLEAS through: (i) 23 force-aligned integration tests passing in under one second with zero application programming interface (API) calls, (ii) a 24-task biomedical benchmark scored by blinded multi-judge evaluation in which BioClaw (PLEAS on OpenClaude) achieves 42/48 rubric points versus 39/48 for a non-PLEAS baseline and 31/48 for GPT-5.5-thinking-extended (blinded), (iii) a single-run token comparison showing 65.4% fewer input tokens with PLEAS active, and (iv) a health-economics case study demonstrating graceful failure and structured artifact generation as architectural properties of the Learn and Share phases. All four validation components are preliminary and their scope is explicitly characterized.

Keywords—artificial intelligence (AI), reusable artifacts, scientific AI workflow, AI agents.

I. INTRODUCTION

Scientific artificial intelligence (AI) workflow agents should not only plan, but also execute, and evaluate. They must also learn before execution and share after assessment. This is the central architectural argument of this paper, and it rests on a structural observation: current agent frameworks leave two phases unspecified. A well-documented tension runs through the design of autonomous agents for scientific workflows. Task flexibility demands that agents reason over open-ended research questions, invoke heterogeneous external tools, and adapt their approach based on intermediate results. Operational soundness demands that agents remain predictable, context-efficient, and reproducible — properties that the field has inherited from decades of scientific workflow engineering in systems such as Galaxy [1], Nextflow [2], and the Common Workflow Language [3].

Current large language model (LLM)-based agent frameworks resolve this tension primarily through iteration: a planning step generates a task decomposition, execution invokes tools, evaluation assesses outputs, and the cycle repeats until the

task is complete or a budget is exhausted. This Plan–Execute–Evaluate loop, embodied in systems from ReAct [10] to Plan-and-Solve [11] to AutoGen [12], is powerful but carries a structural cost. Domain-relevant knowledge is acquired reactively, through tool calls that may fail or return incomplete information, inflating context without proportionate gain. Results synthesized during a run are not externalized in a reusable form; the next agent invocation starts from scratch on the same retrieval problem. As task complexity grows, these costs compound.

We argue that these costs reflect two architectural gaps: the lack of a dedicated knowledge retrieval phase before execution, and the lack of a structured externalization phase after assessment. A Learn phase positions knowledge structuring before any tool is invoked. A Share phase externalizes findings as structured, citable artifacts that persist beyond session end. Together, these additions complete a five-phase specification we call PLEAS (Plan, Learn, Execute, Assess, and Share).

PLEAS is designed as a substrate-independent contract: it specifies phase semantics and structural properties that any compliant execution environment must satisfy, without prescribing implementation. We instantiate PLEAS on two production substrates — Biomni [7] and OpenClaude [8] — and demonstrate a preliminary portability probe on LangGraph [9]. Five structural forces, formally defined and mechanically tested, produce a context-bound independent of session length.

This paper makes four contributions. First, we formalize the PLEAS specification with five structural forces, three portability invariants, and a formal context bound. Second, we report 23 force-aligned integration tests validating all five forces with zero application programming interface (API) calls. Third, we present a 24-task biomedical scientific workflow benchmark with a three-system comparison and a single-run token-efficiency analysis. Fourth, we present a health-economic case study demonstrating graceful failure and artifact generation as architectural properties of the Learn and Share phases.

II. BACKGROUND AND RELATED WORK

A. Scientific Workflow Systems

Galaxy [1] provides a graphical user interface (GUI)-accessible workflow manager with strong provenance tracking for bioinformatics and broader scientific pipelines. Nextflow [2] and Snakemake [4] enable scalable computational workflows through domain-specific languages with implicit dependency management. The Common Workflow Language [3] offers a portable workflow description format that decouples pipeline logic from execution environment. The W3C PROV Data Model [5] and the Taverna Workbench [6] established data-

This work was supported in part by UAB startup funding to JYC and NIH grants to JYC: U54-OD036472 and UM1TR004771.

provenance standards for tracking entity, activity, and agent relationships through computational experiments. PLEAS differs from these systems in targeting adaptive, question-driven workflows, in which the execution plan is produced by an AI agent rather than specified in advance by a researcher. The F3 typed-contract force partially implements phase-level provenance; full alignment with W3C PROV is identified as future work.

B. LLM Agent Architectures

The ReAct framework [10] interleaves reasoning traces and tool-use actions in a single-phase loop, establishing the Plan–Execute–Evaluate pattern now common across agent frameworks. Plan-and-Solve [11] separates high-level planning from step-by-step execution but retains reactive knowledge acquisition. AutoGen [12] introduces multi-agent communication as a coordination primitive but does not formalize phase boundaries or context management. None of these frameworks includes a dedicated knowledge retrieval phase before tool invocation, nor a structured externalization phase after task completion. PLEAS contributes precisely these two additions, alongside a formal context bound derived from five architectural forces.

C. Context Management in Long-Horizon AI Agents

Context window pressure in long-horizon AI agents has been identified as a primary bottleneck for complex task performance; memory-management systems such as MemGPT [13] address this by implementing hierarchical paging of

information into and out of the active context window. PLEAS addresses context pressure architecturally rather than through session-level heuristics: the five structural forces guarantee that context per phase boundary is bounded regardless of session history length. This bound applies to active context passed across phase transitions, not to cumulative tokens consumed over the full run; total token cost grows with the number of tool calls, but no single phase inherits an unbounded history.

III. THE PLEAS SPECIFICATION

A. Phase Contract

PLEAS defines five ordered phases with strict sequencing constraints (**Fig. 1**). Each phase receives a typed input, produces a typed output validated at the phase boundary (Structural Force F3), and writes to phase-specific state fields not accessible to non-adjacent phases (F2). The five phases are:

1) **Plan**. Receives the task specification. Produces a typed plan object — a JavaScript Object Notation (JSON) document with decisions, assumptions, risks, executable steps, and success criteria — that serves as the contract for all subsequent phases. No external system calls are permitted.

2) **Learn**. Receives the plan object. Structures the retrieval strategy, evidence schema, trusted-source registry, and cached or prior contextual knowledge before any live external call is made. The Learn phase transforms reactive knowledge discovery into proactive knowledge structuring: it specifies what to retrieve and in what form, then records retrieval targets as part of its typed output. Live retrieval calls are performed by

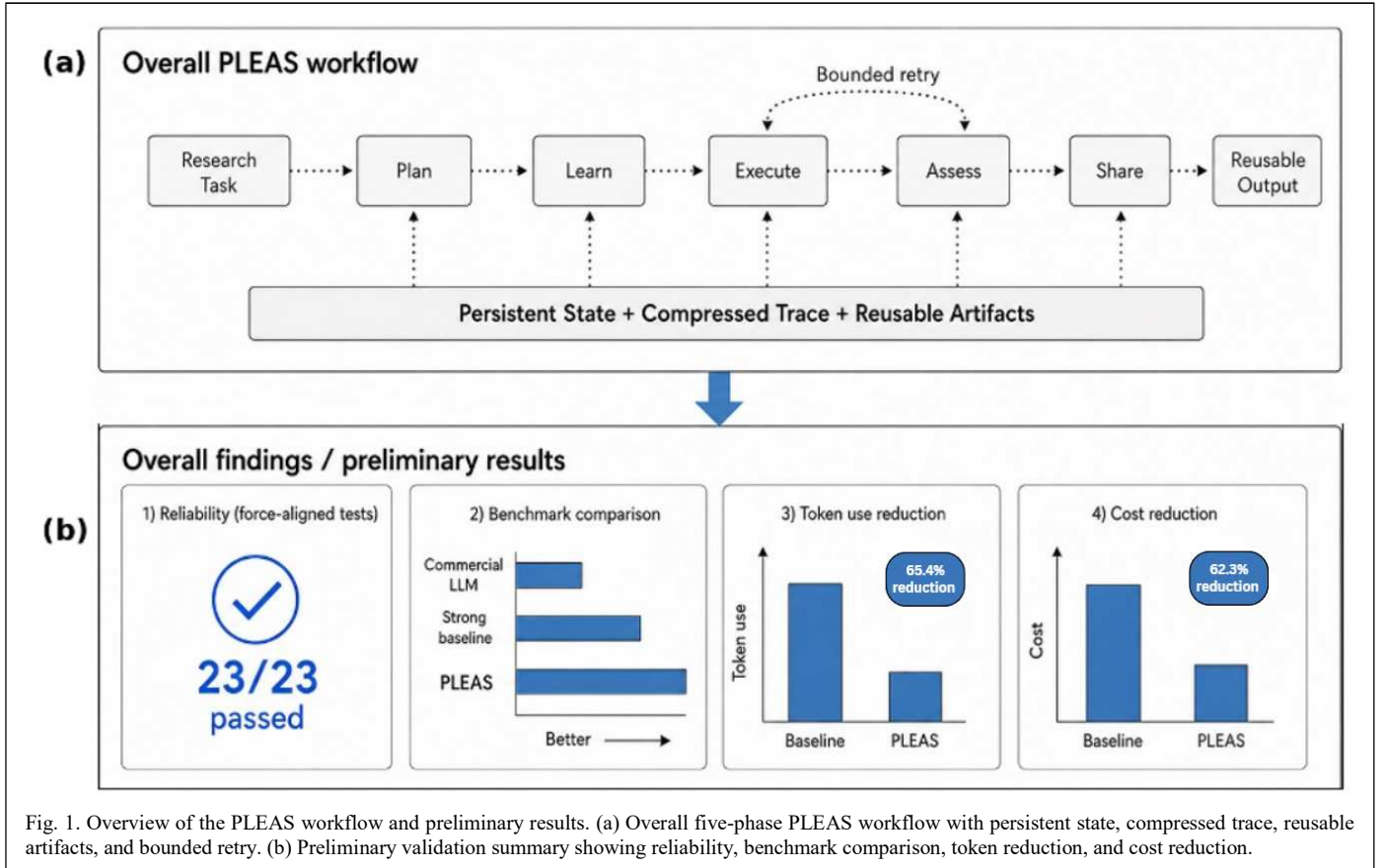


Fig. 1. Overview of the PLEAS workflow and preliminary results. (a) Overall five-phase PLEAS workflow with persistent state, compressed trace, reusable artifacts, and bounded retry. (b) Preliminary validation summary showing reliability, benchmark comparison, token reduction, and cost reduction.

Execute, so that the boundary between knowledge planning and knowledge acquisition is architecturally explicit.

3) **Execute**. Receives plan and evidence schema. This is the only phase permitted to contact external systems — APIs, databases, code executors, and search engines. Execution is bounded by the F4 retry limit (at most $R_{\max} = 3$ attempts per step) and the F5 trace budget (at most $S_{\max}$ tokens per phase summary). Exit codes and artifacts are recorded in typed execution records.

4) **Assess**. Receives plan, evidence, and execution records. Evaluates outputs against plan success criteria. If criteria are not met, Assess triggers a bounded retry back to Execute rather than returning an unchecked result, preventing unconstrained iteration.

5) **Share**. Receives all prior phase outputs. Produces a structured, human-readable artifact — a Markdown report, JSON evidence record, or annotated code repository — with full provenance tracing. Share ensures that synthesized knowledge is externalized for downstream reuse rather than discarded at session end.

B. Five Structural Forces

Five design properties jointly bound context growth across a PLEAS run. Force F1 (Context Boundedness) is emergent rather than directly engineered; it arises as a corollary of F2 through F5. *Table I* summarizes all five forces. **Corollary 1 (Context Bound)**. Under fixed $S_{\max}$ and $R_{\max}$: $\text{context}(i) \leq 2*S_{\max} + K*B + T + A$, where K = Execute tool calls, B = average bounded tool-response size, T = plan object size, A = schema overhead. This bound is $O(1)$ with respect to session length. It applies to active context passed at phase boundaries, not to cumulative tokens consumed over the entire run. A Plan–Execute–Evaluate loop without these forces grows as $O(n*B)$ in the number of tool calls n .

C. Portability Invariants

Three invariants define substrate compliance. P1 (Portability): The five-phase semantics are preserved unchanged under substrate migration. P2 (Composability): any individual phase may be substituted by a substrate-specific implementation without altering adjacent phase contracts. P3 (Domain-Extensibility): domain-specific tools can be injected at Execute without modifying the orchestrator.

TABLE I. FIVE STRUCTURAL FORCES

Force	Name	Enforcement role
F1	Context boundedness	Emerges from F2-F5; active phase context is bounded.
F2	Phase-scoped execution	Each phase receives only required inputs; raw history is pruned.
F3	Typed contracts	Schemas validate inputs/outputs before state mutation.
F4	Bounded retry	Execute retries at most $R_{\max}=3$; failures pass to Assess.
F5	Trace compression	Summaries are capped at $S_{\max}$ and stored in long-term memory.

IV. PLEAS INSTANTIATIONS

A. PLEAS on Biomni

Biomni [7] is a Python-native scientific agent platform with a rich tool registry (PubMed, ChEMBL, UniProt, ClinVar, RCSB PDB, the Human Cell Atlas, and others) and Pydantic-typed tool schemas. PLEAS maps each phase to a Biomni orchestration wrapper around the native tool-dispatch loop. Every phase boundary enforces a Pydantic schema check via F3; outputs that fail validation trigger a corrective retry within the $R_{\max}$ budget. Phase-exit compression (F5) reduces the log to at most $S_{\max} = 2,000$ tokens before the next phase inherits it. The budget dictionary — tokens, seconds, and USD — is tracked independently across all three dimensions.

B. BioClaw on OpenClaude

BioClaw re-instantiates PLEAS on OpenClaude [8], which uses SKILL.md phase files and Model Context Protocol (MCP) tool schemas rather than Biomni's Python registry. The five-phase semantics are preserved; substrate-specific differences are confined to the Execute adapter (MCP tool schemas) and the Learn retriever (SKILL.md knowledge files). This localization of substrate differences to adapter code is the concrete expression of P1 and P2.

C. LangGraph Portability Probe

We implemented a three-phase PLEAS stub on LangGraph [9] to probe P1 on a structurally different framework. The compression middleware — `_roll_memory()` and `_prune_history()`, totaling 18 lines of core logic — was inserted at each phase transition to enforce F5 and F1. Plan and Assess phases ran without modification. Execute required a ToolNode adapter of approximately 18 lines to translate PLEAS Pydantic schemas into LangGraph's tool-dispatch interface. The Learn and Share phases — the two architecturally novel phases — were not tested in this probe. We therefore report the result as a feasibility observation rather than a portability demonstration.

V. VALIDATION

A. Force-Aligned Integration Tests

We implemented 23 force-aligned integration tests (IT-F suite) covering all five structural forces. All 23 pass in 0.95 seconds with zero application programming interface (API) calls, using lightweight stubs for all LLM providers and external dependencies. *Table II* summarizes coverage; the full test file (`test_it_forces.py`) with reproduction instructions is provided in the supplemental material. These tests validate structural enforcement under stub implementations and do not validate live API behavior, tool correctness, or domain accuracy.

TABLE II. IT-F INTEGRATION TEST RESULTS.

Force	Tests	Count	Behavior
F1	IT-F1-01-04	4	State isolation; per-phase directories.
F2	IT-F2-01-06	6	Schema accept/reject payloads.
F3	IT-F3-01-05	5	Plan schema guards; empty-step rejection.
F4	IT-F4-01-04	4	Retry counter; budget-exhaustion stop.
F5	IT-F5-01-04	4	Long-term memory; JSON round-trip.
Total	-	23	23/23 PASS; 0.95 s; 0 API calls.

Representative examples illustrate force coverage. IT-F1-03 verifies that StateManager writes PLAN and EXECUTE outputs to separate filesystem subdirectories. IT-F2-02 verifies that EvidenceSchema raises a Pydantic ValidationError when the required source_id field is absent. IT-F4-03 verifies that execute() runs no plan steps when the seconds budget equals zero. IT-F5-02 verifies that a long_term_memory string survives a JSON serialization round-trip without loss.

B. 24-Task Biomedical Scientific Workflow Benchmark

We designed a 24-task biomedical scientific workflow benchmark spanning six categories: Sequence & Variant Analysis, Drug-Target Discovery, Literature & Clinical Mining, Transcriptomics & Single-Cell Analysis, Structural & Protein Biology, and Pathway & Systems Biology. Tasks span easy (11), medium (9), and hard (4) difficulty tiers. Each task uses a 0/1/2 rubric: score 2 requires a correct factual answer, correct database or tool, and explicit reasoning; score 1 indicates a correct approach with incomplete output; score 0 is a failure. Task text, rubric, and raw outputs are in the supplemental.

Three systems were evaluated: BioClaw (PLEAS on OpenClaude), Biomni without PLEAS orchestration (non-PLEAS baseline), and GPT-5.5-thinking-extended. Scoring used a blinded multi-judge protocol: responses were anonymized via HMAC-SHA256 with system-identifying names scrubbed by regex, then independently scored by three LLM judges (Claude Opus 4.8, Claude Sonnet 4.6, Gemini 2.5 Pro). Majority-vote aggregation yielded per-task scores; interrater reliability measured Fleiss' kappa of 0.53 (moderate agreement) and Krippendorff's alpha of 0.11. Per-task judge-level scores are provided in the supplemental materials. **Fig. 2** presents results.

BioClaw achieved 42/48 rubric points (87.5%), the non-PLEAS baseline achieved 39/48 (81.2%), and GPT-5.5-thinking-extended achieved 31/48 (64.6%). Transcriptomics & Single-Cell was the most discriminating category: BioClaw scored 7/8, baseline 6/8, and GPT only 2/8, suggesting structured knowledge retrieval provides the greatest advantage

on multi-step analytical tasks. The 3-point margin over the non-PLEAS baseline must be interpreted cautiously because BioClaw and Biomni differ in both orchestration and model routing. BioClaw was run through OpenClaude using MiniMax M2.7, whereas the non-PLEAS Biomni baseline used higher-tier model routing; therefore, the observed BioClaw margin is unlikely to be explained by use of a stronger frontier model, but it still cannot be attributed solely to PLEAS without a fully matched controlled run.

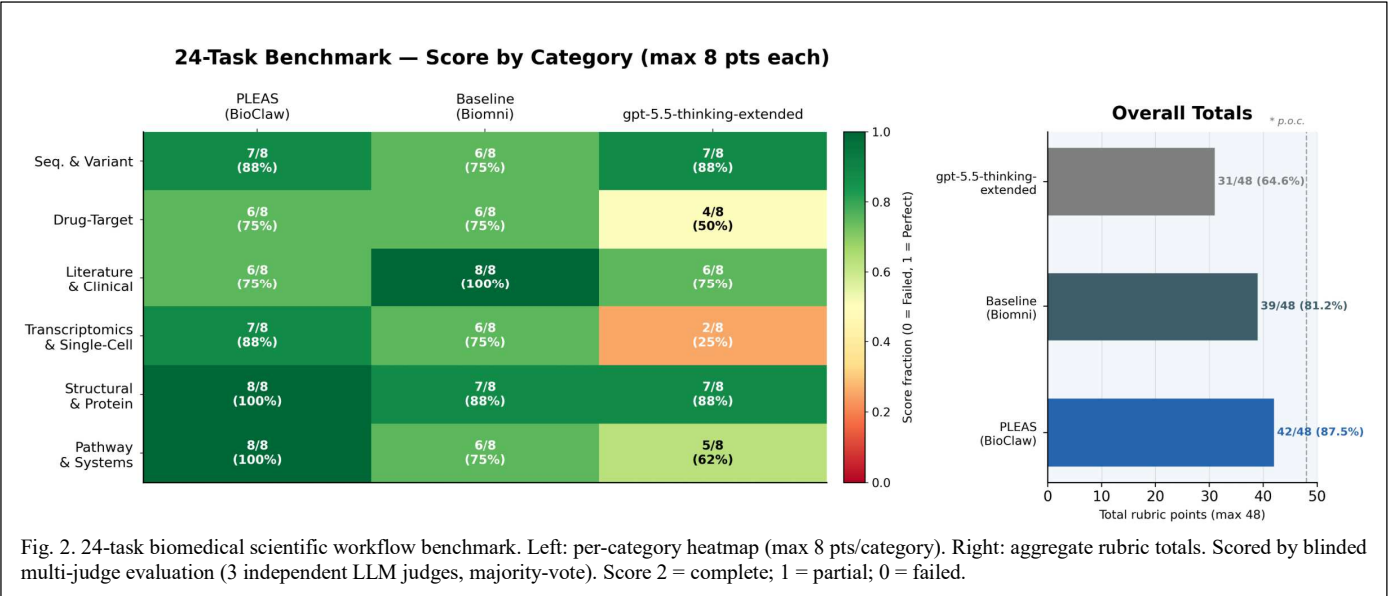
C. Token Efficiency Analysis

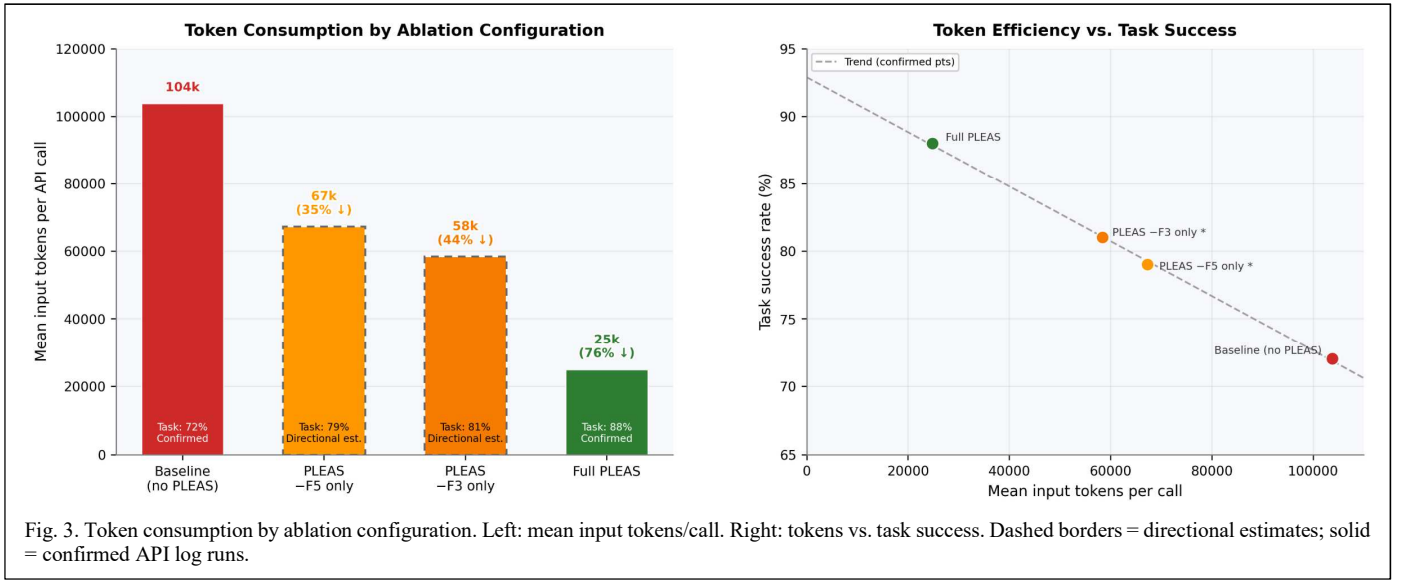
We measured input token consumption across one end-to-end run per condition: (i) a non-PLEAS Biomni run (18 API calls, September 2025) and (ii) a full PLEAS run (18 API calls, June 2026). Both runs use confirmed API log data; call-by-call counts are in Supplemental Table S2. The PLEAS run consumed 646,825 total input tokens versus 1,868,947 for the baseline — a 65.4% reduction. Average tokens per call fell from 103,830 to 35,935. This is a descriptive single-run comparison; the two runs differ in date, task, and model availability, so no statistical inference is warranted.

A directional ablation study deactivated individual forces. Removing F5 (Trace Compression) increased mean tokens to approximately 67,320/call; removing F3 (Phase Isolation) increased mean tokens to approximately 58,441/call. These intermediate configurations were not independently re-run and are marked as directional estimates in **Fig. 3**. The confirmed endpoints — Baseline (103,830/call) and Full PLEAS (35,935/call) — support the architectural prediction that F3 and F5 each contribute to context reduction; quantifying their interaction requires controlled re-runs. **Fig. 4** shows the call-by-call view: both conditions increase over time, but the baseline grows rapidly (24K→195K tokens) as context accumulates, while BioPLEAS grows slowly (20K→52K) because phase isolation and trace compression bound the growth rate.

D. Failure Analysis

Under blinded multi-judge scoring, GPT-5.5-thinking-extended received four zero scores (T07, T14, T16, T23), all in





categories requiring multi-step tool orchestration; neither BioClaw nor the non-PLEAS baseline received any zero. T03 received a score of 1 from all three systems, suggesting rubric ambiguity rather than capability differences. T14 (GSE159929 dataset mismatch) was correctly identified by BioClaw (score 2) but failed under GPT, indicating structured retrieval planning in the Learn phase improves error detection.

E. Case Study: Health-Economic Incremental Cost-Effectiveness Ratio (ICER) Workflow

The health-economic ICER case study evaluates PLEAS on an open-ended task requiring evidence synthesis, economic modeling, and structured result sharing for adolescent sleep-disorder interventions.

Plan. PLEAS decomposed the task into evidence gathering, effect-size synthesis, ICER computation with sensitivity analysis, and decision synthesis, with criteria and fallbacks fixed before tool use.

Learn. The Learn phase defined the intervention taxonomy, target databases, and extraction schema before Execute made live external calls.

Execute. Execute ran searches and generated ICER and probabilistic-sensitivity code; unit tests found and corrected a formula bug within the bounded retry budget, and market-price surveys were completed.

Assess. The assessment found that only 4 of 18 planned interventions had valid evidence-registry entries and returned a structured failure rather than fabricating effect-size inputs.

Share. Share produced a draft report documenting completed analyses, evidence-retrieval failures, and a remediation roadmap, showing that artifact generation persists even after primary-task failure.

The trace shows typed plan decomposition, graceful failure, and persistent artifact generation as consequences of the PLEAS phase contract.

VI. DISCUSSION

A. Evidence Calibration

Table III maps each empirical claim to its supporting evidence and characterizes the scope. The integration tests are the strongest evidence: they are repeatable, deterministic, and require no external resources. The benchmark, token, portability, and ablation results are preliminary.

B. Evaluation Scope and Limitations

Two limitations bound interpretation: one token run per condition with unmatched model routing, and an incomplete portability probe. Blinded multi-judge scoring addresses the prior non-blinded evaluation confound. The benchmark provides a directional estimate of the Learn and Share contribution: the non-PLEAS baseline, which omits both phases, scored 39/48 versus BioClaw's 42/48, a 3-point gap concentrated in categories requiring multi-step tool orchestration. Matched token runs and full five-phase ports are needed before strong claims can be made.

Although the benchmark uses biomedical tool registries, the PLEAS contract is domain-agnostic: it specifies phase semantics rather than tool content. Future work should test at least one non-biomedical scientific domain, run controlled multi-task token experiments, add blinded human scoring, port all five phases to additional substrates, and extend the benchmark beyond biomedicine.

TABLE III. EVIDENCE-TO-CLAIM MAPPING

Claim	Evidence and scope
F1-F5 enforce bounded context	23/23 IT-F tests plus formal context-bound derivation; live API behavior not tested.
65.4% token reduction	One API-log run per condition; descriptive, not controlled.
87.5% benchmark score	24 biomedical tasks; blinded multi-judge scoring (Fleiss' kappa 0.53)
LangGraph portability probe	Three-phase stub; Learn and Share remain untested.
Graceful failure and sharing	Single ICER case study; qualitative evidence only.

Call-by-Call Input Token Consumption (one end-to-end run per condition, confirmed from API logs)

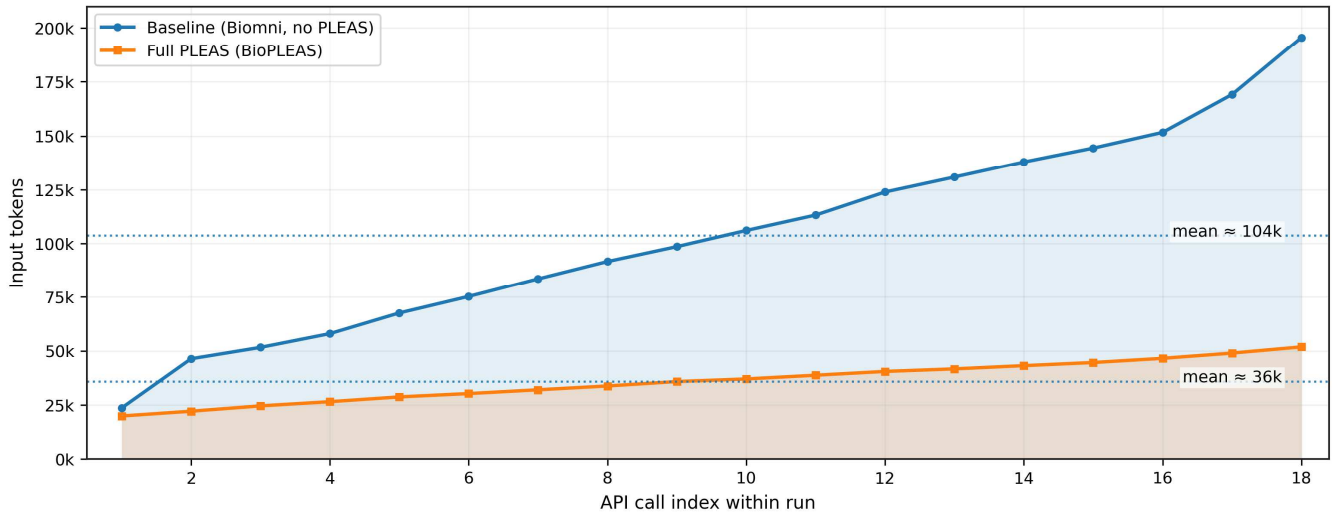


Fig. 4. Call-by-call input token counts (Claude 4.5 Sonnet). The baseline increases over time as accumulated context expands across API calls; BioPLEAS grows slowly (20K→52K) under phase isolation and trace compression.

VII. CONCLUSION

Scientific artificial intelligence (AI) workflow agents must learn before execution and share after assessment. These two requirements, missing from the prevailing Plan–Execute–Evaluate loop, are the central argument of this paper. PLEAS (Plan, Learn, Execute, Assess, Share) provides the five-phase specification that realizes them. Five structural forces produce a context-bound state, as stated in Corollary 1, applying to active context at phase boundaries, not cumulative tokens, that is independent of session length. Twenty-three force-aligned integration tests confirm all five forces are mechanically enforced in PLEAS on Biomni, passing in 0.95 seconds with zero application programming interface (API) calls. A 24-task biomedical scientific workflow benchmark — scored by blinded multi-judge evaluation — shows BioClaw achieving 42/48 rubric points versus 39/48 for a non-PLEAS baseline and 31/48 for GPT-5.5-thinking-extended (blinded). A single-run token comparison shows a 65.4% reduction consistent with the architectural predictions. A health-economics case study demonstrates that the Learn and Share phases produce graceful failure and artifact generation as architectural properties rather than emergent model behaviors.

The specification, integration tests, middleware extract, and benchmark data are provided as supplemental material. The primary contribution is the PLEAS specification and validated structural forces; the empirical results are early evidence for future replication. Code and data availability: The PLEAS specification, BioClaw reference implementation, 24-task benchmark, and blinded scoring pipeline are publicly available at <https://github.com/alphamind-club/PLEAS>.

REFERENCES

- [1] E. Afgan et al., “The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update,” *Nucleic Acids Research*, vol. 46, no. W1, pp. W537–W544, 2018, doi: 10.1093/nar/gky379.
- [2] P. Di Tommaso et al., “Nextflow enables reproducible computational workflows,” *Nature Biotechnology*, vol. 35, no. 4, pp. 316–319, 2017, doi: 10.1038/nbt.3820.
- [3] P. Amstutz et al., “Common Workflow Language, v1.0,” *figshare*, 2016, doi: 10.6084/m9.figshare.3115156.v2.
- [4] J. Köster and S. Rahmann, “Snakemake—a scalable bioinformatics workflow engine,” *Bioinformatics*, vol. 28, no. 19, pp. 2520–2522, 2012, doi: 10.1093/bioinformatics/bts480.
- [5] L. Moreau and P. Missier, Eds., “PROV-DM: The PROV Data Model,” *W3C Recommendation*, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>. Accessed: May 16, 2026.
- [6] K. Wolstencroft et al., “The Taverna workflow suite: designing and executing workflows of Web Services on the desktop, web or in the cloud,” *Nucleic Acids Research*, vol. 41, no. W1, pp. W557–W561, 2013, doi: 10.1093/nar/gkt328.
- [7] K. Huang et al., “Biomni: A general-purpose biomedical AI agent,” *bioRxiv*, 2025, doi: 10.1101/2025.05.30.656746.
- [8] Gitlawb, “OpenClaude,” *GitHub repository*. [Online]. Available: <https://github.com/Gitlawb/openclaude>. Accessed: May 16, 2026.
- [9] LangChain AI, “LangGraph,” *GitHub repository*. [Online]. Available: <https://github.com/langchain-ai/langgraph>. Accessed: May 16, 2026.
- [10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.
- [11] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 2609–2634, doi: 10.18653/v1/2023.acl-long.147.
- [12] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv:2308.08155*, 2023.
- [13] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” *arXiv:2310.08560*, 2023.